

Database Fundamentals Project — Event Management App

1. Project goal

We are building an Event Management App for the Database Fundamentals course. The project focuses mainly on the database design, schema, queries, and integration into a simple real application, which matches the course project requirements.

2. Course project requirements

Minimum required deliverables

- Entities description and most critical scenarios / common user paths
- Domain description
- Database schema in SQL and DBML
- Database schema image
- Some fake but valid data
- 3 SQL queries to the database
- Indexes / performance optimizations where needed

Recommended for a higher score

- everything above
- a working MVP at least locally, with a recorded demo

- ideally a deployed app with database, backend, and frontend hosted in the cloud

Deadline

- Project deadline: May 30

Team

- Groups of maximum 3 students are allowed
-

3. Chosen topic

We chose the Event Management App, which is one of the suggested course project ideas. The project sheet also already proposes 3 good SQL tasks for this domain:

1. count total attendees for each event
 2. calculate total revenue generated by ticket sales for each organizer
 3. find the average attendance rate for events in the past 3 months
-

4. Scope of our version

We are keeping the project focused and realistic.

Main functionality

- organizers can create and manage events

- users can browse events
- users can book tickets
- payments are recorded
- attendees can be checked in
- organizers can monitor attendance and revenue

We are NOT adding unnecessary features

We are not trying to build a huge product with chat, social feed, recommendations, maps, etc. The focus is the database.

5. Domain description

The Event Management App is a platform for organizing and managing events such as conferences, workshops, networking sessions, and student activities. Organizers can create events, define ticket types, and monitor registrations. Users can browse available events, book tickets, make payments, and attend events. The system stores information about organizers, venues, events, ticket categories, bookings, payments, and attendee check-ins. The database must support event creation, ticket booking, payment tracking, revenue analysis, and attendance monitoring.

This description should be detailed enough that someone could understand the field and reconstruct the core database structure from it.

6. Critical scenarios / common user paths

These are the main user paths we should include in the report:

1. An organizer creates an account.
2. The organizer creates an event.

3. The organizer selects a venue and defines one or more ticket types.
4. A user browses available events.
5. A user books one or more tickets for an event.
6. A payment record is created for the booking.
7. On event day, attendees are checked in.
8. The organizer reviews attendance and revenue statistics.

These are the most important flows for the database design.

7. Main entities

We should use these entities:

7.1 Users

People who browse events and make bookings.

7.2 Organizers

Individuals or organizations responsible for creating and managing events.

7.3 Venues

Physical places where events are hosted.

7.4 Events

The events themselves, such as conferences, workshops, and seminars.

7.5 TicketTypes

Different ticket categories for an event, such as General Admission, Student, or VIP.

7.6 Bookings

Reservations made by users for event tickets.

7.7 Payments

Payment records linked to bookings.

7.8 CheckIns

Attendance confirmation for booked users at the event.

8. Main relationships

- one organizer can create many events
 - one venue can host many events
 - one event can have many ticket types
 - one user can make many bookings
 - one ticket type can be booked many times
 - one booking can have one payment
 - one booking can have zero or one check-in
-

9. Proposed schema draft

organizers

- organizer_id
- organizer_name
- email

- phone
- created_at

users

- user_id
- full_name
- email
- phone
- created_at

venues

- venue_id
- venue_name
- address
- city
- capacity

events

- event_id
- organizer_id
- venue_id

- event_name
- description
- category
- start_datetime
- end_datetime
- status
- created_at

ticket_types

- ticket_type_id
- event_id
- ticket_name
- price
- quantity_available

bookings

- booking_id
- user_id
- ticket_type_id
- quantity
- booking_date

- booking_status

payments

- payment_id
- booking_id
- amount
- payment_method
- payment_status
- payment_date

check_ins

- check_in_id
- booking_id
- check_in_time
- checked_in_by

10. Suggested SQL queries

These should be part of the final submission because the project sheet explicitly asks for 3 queries and suggests strong examples for this domain.

Query 1

Count the total number of attendees for each event.

Query 2

Calculate the total revenue generated by ticket sales for each organizer.

Query 3

Find the average attendance rate for events in the past 3 months.

11. Fake data plan

We should generate fake but valid data for:

- 5 organizers
- 15 users
- 5 venues
- 10 events
- 20 ticket types
- 30 to 50 bookings
- payments for the bookings
- check-ins for some completed bookings

This should be enough to make the database look realistic and allow the SQL queries to return meaningful results.

12. Indexes / optimization ideas

We should add indexes on important foreign keys and frequently queried columns, for example:

- events.organizer_id
- events.venue_id
- ticket_types.event_id
- bookings.user_id
- bookings.ticket_type_id
- payments.booking_id
- check_ins.booking_id

These can be mentioned in the report as performance optimizations.

13. Tech stack idea

The project sheet recommends:

- Frontend: React
- Backend: FastAPI (Python)
- Database: PostgreSQL
- optional ORM such as SQLAlchemy

For deployment, the sheet suggests tools like:

- Supabase
- Render
- Heroku

- PythonAnywhere
- Netlify
- Vercel
- GitHub Pages

For now, we should first complete the minimum database requirements before worrying about deployment.

14. Work split proposal

My part

- domain description
- critical scenarios / user paths
- entity descriptions
- DBML draft
- schema image
- report writing

Teammate part

- SQL DDL
- fake data insertion
- 3 SQL queries
- indexes / optimization section

- optional MVP / backend / demo

Shared

- review all parts together
 - make sure naming is consistent
 - prepare final submission
-

15. Immediate next steps

1. finalize the list of entities
 2. create the DBML schema
 3. generate the schema image
 4. write the SQL DDL
 5. add fake data
 6. write the 3 required SQL queries
 7. add indexes
 8. decide whether we also do a simple MVP/demo
-

16. Important note

The project should stay simple, clean, and database-focused. A smaller correct project is much better than an overambitious messy one.

